

cs-mail Protocol Specification

Permission, Bonded Contact, and Relationship Settlement

C-SQD

Draft 0.5 — August 2026

Abstract

This document defines the normative state, commands, events, settlement rules, error conditions, and invariants of the *cs-mail* protocol. The protocol represents communication permission as a directed relationship between public protocol identities. An unaccepted sender reserves a bond before attempting contact. Acceptance and rejection are relationship-level decisions that atomically settle every eligible admitted bond and fully cancel every unadmitted reservation for that relationship. Rejection declines one solicitation while preserving the possibility of a later attempt after backoff; a separate recipient block prohibits future ordinary contact until removed. The specification also distinguishes reservation from admission: an abandoned unadmitted reservation returns in full, and the recipient decision window begins only when the message is admitted. Repeated-attempt controls remain scoped to the private principal and are not reset by accepting one public identity. The specification further defines persistence reserves, expiry ordering, and the coalescing of repeated recipient solicitations. It separates protocol settlement, ledger operation, and external financial rails. It also defines a native privacy profile covering end-to-end content encryption, pairwise-scoped identifiers, compartmentalized protocol data, limited retention, and cryptographic erasure.

1 Status and Conformance

This is an initial protocol draft. It is intended to make the deterministic core implementable and testable; it does not select a wire encoding, signature suite, database, transport, banking partner, or jurisdiction-specific compliance model.

The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** in this document describe conformance requirements. An implementation conforms to this version only if every externally visible transition preserves the state and financial invariants defined here.

2 Scope

This specification defines:

- private principals and public protocol identities;
- directed communication relationships;
- signed contact terms;
- bonded communication attempts and persistence reserves;
- relationship acceptance, rejection, blocking, unblocking, and revocation;

- unadmitted reservation cancellation and admitted-bond expiry;
- repeated-attempt admission and principal-level continuity;
- relationship-solicitation coalescing;
- canonical event ordering, idempotency, and atomic settlement; and
- native content confidentiality, relationship-graph privacy, record lifecycle, minimum ledger semantics, and protocol invariants.

Message-body formats, content moderation, inbox ranking, user-interface design, external funding, withdrawal, foreign exchange, and inter-provider net settlement are outside the deterministic kernel. Those systems **MUST NOT** contradict the state and settlement consequences defined here.

3 Protocol Model at a Glance

Status of this section. This section is nonnormative. It gives the reader a map of the interacting state before the following sections define it precisely. If this overview conflicts with a later requirement, the later normative requirement controls.

3.1 One First Contact

Alice’s public protocol identity has not been accepted by Bob’s public protocol identity. Bob’s provider quotes $C = 0.25$ and $S = 4.75$ settlement units. On the first attempt, $L_0 = 0$, so Alice reserves $R_0 = 5.00$. After the encrypted message is admitted, Bob’s relationship decision determines both permission and money:

Outcome	Bond settlement	Relationship effect	Repeated-attempt effect
Accept	Return $C + S$ to Alice.	Establish directed permission for Alice’s public identity.	Release active L_k for that identity; preserve principal-level history.
Reject	Transfer C to Bob’s provider and S to Bob.	Decline this solicitation; permit a later eligible attempt after backoff.	Preserve the advanced level; release each L_k on schedule.
Expire	Transfer C to Bob’s provider and return S to Alice.	Leave the current relationship state unchanged.	Preserve the advanced level; release each L_k on schedule.

Acceptance and rejection apply to the Alice-to-Bob relationship, not merely to the message beside which Bob acts. A timely decision therefore settles every eligible admitted bond for that directed identity pair. A reserved but unadmitted message is instead cancelled with a full return. The detailed definitions appear below. The first admitted attempt opens one relationship solicitation; later attempts join that grouped episode without generating another relationship- establishment signal.

3.2 Five Interacting Lifecycles

The protocol composes five related but distinct lifecycles:

Lifecycle	States or stages	What changes it
Directed relationship	unknown, accepted, rejected, revoked, blocked	Recipient acceptance, rejection, revocation, blocking, or unblocking.
Communication attempt	terms issued, value reserved, then admitted or cancelled	Sender authorization followed by durable admission, sender cancellation, or admission timeout.
Bond	reserved or admitted , then one terminal state	Unadmitted cancellation, acceptance, rejection, blocking, or expiry.
Persistence reserve	reserved , then released exactly once	Immediate release on unadmitted cancellation or identity-scoped acceptance; scheduled release after rejection, blocking, or expiry.
Relationship solicitation	inactive, active generation, then closed	First admitted unaccepted attempt opens it; later attempts join it; a relationship decision or exhaustion of decision-open bonds closes it.

The relationship controls future communication. The bond settles one or more pending attempts. Repeated-attempt state controls when and with how much additional refundable liquidity another attempt may be admitted. The relationship solicitation controls whether the recipient receives a new request to establish the relationship or merely sees another attempt grouped into an existing request. The ledger records the ownership consequences. No one state is a substitute for another.

3.3 Identity and Reference Layers

A private principal P_i is a provider-local control and anti-abuse subject. A public protocol identity I_i is the visible persona to which permission is granted. An opaque relationship handle H_{ij} represents a directed identity pair inside a limited service domain. Ledger, content, and telemetry systems use their own scoped references. These layers deliberately do not collapse into one global user identifier.

4 Terminology and Actors

4.1 Actors

Sender

An authenticated private principal acting through a public protocol identity to initiate a communication attempt.

Recipient

The private principal controlling the destination protocol identity and its directed communication relationship.

Sender provider

The service that authenticates the sender and reserves sender funds.

Recipient provider

The service that publishes contact terms, admits the message, records recipient decisions, and delivers to the recipient.

Canonical journal

The authoritative, append-only ordering of protocol and ledger events for a deployment or federated settlement domain.

Ledger operator

The service maintaining canonical reservations, transfers, releases, and balance projections.

One organization **MAY** perform several roles in a centralized deployment. Their logical responsibilities remain distinct.

4.2 Identifiers

The following notation is used:

- P_i : the private principal reference for sender i within its authoritative identity service;
- I_i : sender i 's public protocol identity;
- I_j : recipient j 's public protocol identity;
- A_{ij} : permission state from I_i to I_j ; and
- (P_i, I_j) : the private key for repeated-attempt state.

The notation P_i denotes the continuing enforcement subject, not a portable or globally comparable identifier. An implementation **MUST NOT** disclose its local principal reference or underlying identity evidence in ordinary messages, public relationship records, or inter-provider envelopes. A federated assertion **MAY** carry an opaque, scoped reference sufficient to address the correct repeated-attempt state.

4.3 Relationship-Level Decision

An acceptance, rejection, or block applies to a directed communicative relationship, not to one selected message. A user interface **MAY** place the action beside a message, but the authenticated command **MUST** identify the sender and recipient protocol identities and **MUST** carry relationship-level consequences. Rejection declines the active solicitation and preserves the possibility of a later eligible attempt after backoff. Blocking separately prohibits future ordinary contact until the recipient removes the block.

4.4 Relationship Solicitation Episode

A relationship solicitation episode is the recipient-provider grouping of one or more admitted attempts from the same unaccepted sender protocol identity to the same recipient protocol identity. The first admitted attempt in a new episode produces exactly one logical relationship-establishment signal. Its delivery channel may follow recipient settings. Later admitted attempts join the active episode and do not produce another such signal.

The episode is not a message, bond, permission state, or ledger account. Attempts inside it remain distinct and are not idempotent with one another. Each retains its own content, bond, decision deadline, and persistence reserve. Exact replay of one command remains governed by its idempotency key.

Acceptance, rejection, or blocking closes the active episode. Rejection permits a later eligible admission after backoff; blocking does not. Silence is not a relationship command: each bond remains subject to its own expiry. An episode lapses when no associated admitted bond remains decision-open. A later eligible admission may open a new episode. A user-interface dismissal without an authenticated relationship decision has no protocol or settlement consequence.

5 Canonical Time and Event Ordering

Every state-changing command **MUST** have one canonical linearization point in the journal. The journal **MUST** assign a strictly ordered position and an authoritative receipt time. Implementations **MUST NOT** use a sender-controlled or recipient-device clock to determine whether a financial decision was timely.

Commands that appear concurrent are resolved by canonical journal order. A transaction that reads a relationship or repeated-attempt version **MUST** either commit against that version or fail without partial effects.

For an authenticated recipient decision, the decision ingress service **MUST**:

1. authenticate the recipient and the targeted relationship;
2. durably record the command or an equivalent commitment;
3. assign its authoritative receipt time and ordering position; and
4. return a signed decision receipt.

Federated deployments **MUST** define which service is authoritative for decision ingress and how its signed receipts become canonical. A recipient provider **MUST NOT** earn an expiry settlement by discarding a decision for which it issued a valid timely receipt.

6 Protocol State

6.1 Principal and Protocol Identity

A *private principal* is a provider-local authorization and enforcement subject. It binds authentication, control of protocol identities, recovery state, and repeated-attempt continuity. It need not denote a verified human or legal person; provider policy **MAY** use a person, account, organization, or managed group as the subject. A private principal record minimally contains a local, immutable reference, verification status where applicable, authorized public protocol identities, and recovery state.

A *public protocol identity* is a user-visible, protocol-facing persona through which a principal sends and receives communication. Its record minimally contains its identity value, controlling provider, authorized operational keys, status, and version. Directed relationships and communication permission are scoped to these protocol identities.

One private principal **MAY** control multiple public protocol identities. Acceptance of one identity **MUST NOT** imply acceptance of another identity controlled by the same principal. Conversely, changing protocol identities or operational keys **MUST NOT** by itself reset the private principal's repeated-attempt state for the same recipient.

A private principal reference **MUST NOT** serve as a public address, relationship identifier, ledger identifier, telemetry identifier, or globally comparable inter-provider identifier. Outside the authoritative identity service, the principal **MUST** be represented only by purpose-limited, scoped assertions as defined in the privacy requirements of this specification.

6.2 Directed Relationship

The directed relationship state is

$$A_{ij} \in \{\text{unknown}, \text{accepted}, \text{rejected}, \text{revoked}, \text{blocked}\}. \quad (1)$$

The relationship record **MUST** contain I_i , I_j , the current state, a monotonically increasing version, the most recent canonical event position, and the transition time.

A bond is required exactly when the relationship is not accepted and no valid scoped capability authorizes the message:

$$\text{bond_required}(I_i, I_j) = \begin{cases} 0, & A_{ij} = \text{accepted}, \\ 1, & A_{ij} \in \{\text{unknown}, \text{rejected}, \text{revoked}\}. \end{cases} \quad (2)$$

The function is intentionally undefined for **blocked**. A blocked relationship is ineligible for ordinary contact rather than eligible upon payment. The recipient provider **MUST NOT** issue ordinary contact terms or admit an ordinary attempt while the block remains active. A block also overrides previously issued scoped capabilities for the directed pair unless the recipient removes the block through an authenticated transition.

6.3 Repeated-Attempt State

Repeated-attempt state is keyed by (P_i, I_j) , not by a replaceable sender address, operational key, or provider account. It **MUST** contain:

- attempt level k ;
- earliest next-admission time;
- active persistence-reserve references;
- a monotonically increasing version; and
- the most recent canonical event position and transition time.

Changing sender protocol identities, operational keys, or providers **MUST NOT** reset this state. Distinct recipient protocol identities retain distinct state. Acceptance of one public identity **MUST NOT** reset this principal-level state. While that identity remains accepted, its ordinary messages bypass the state; other public identities controlled by the principal remain subject to it. Any decay or forgiveness of old principal-level history **MUST** be an explicit, versioned policy transition rather than an implicit effect of acceptance.

6.4 Contact Terms

Before reserving an unaccepted attempt, the sender **MUST** obtain a signed quote from the recipient provider. A quote **MUST** contain at least:

- quote identifier and protocol version;
- sender and recipient protocol-identity references;
- recipient-provider identity;
- an opaque repeated-attempt-state reference;
- relationship and repeated-attempt versions;
- processing charge C and effective collateral S ;
- attempt level k , backoff eligibility time, and persistence reserve L_k ;
- settlement unit;
- admission-window duration, decision-window duration, and persistence- reserve release conditions;
- quote issue and expiry times;
- privacy-profile and retention-policy versions;
- policy version; and
- provider signature.

The recipient provider establishes C and **MAY** establish minimum collateral $S_{p(j)}^{\min}$. If the recipient preference is $\hat{S}_j$, the effective collateral is

$$S = \max\{\hat{S}_j, S_{p(j)}^{\min}\}. \quad (3)$$

The complete required reservation is

$$R_k = C + S + L_k. \quad (4)$$

Outside the endpoint and routing boundary, the protocol-identity references **MUST** be pairwise-scoped and opaque as defined in the privacy requirements below. The sender **MUST** be shown R_k before authorization. The quote **MUST NOT** add a sender-provider charge, delivery fee, or clearing fee to protocol settlement. Once a reservation is created, its incorporated terms **MUST** remain immutable. The reservation command materializes an admission deadline from the quoted admission-window duration and canonical reservation time. Admission materializes the bond's decision deadline from the quoted decision-window duration and canonical admission time. Both additions **MUST** use checked canonical-time arithmetic.

6.5 Bond

A bond **MUST** contain a unique identifier, quote reference, opaque sender and recipient references sufficient to resolve the directed relationship, attempt and message references, C , S , settlement unit, creation time, admission deadline, optional canonical admission time and decision deadline, current state, and current version. The admission time and decision deadline are absent while the bond remains **reserved** and become immutable when it becomes **admitted**.

Bond state is

$$\text{bond_state} \in \{\text{reserved}, \text{admitted}, \text{accepted}, \text{rejected}, \text{expired}, \text{cancelled_unadmitted}\}. \quad (5)$$

The first two states are nonterminal. The remaining states are terminal. A reserved bond whose admission deadline has passed is cancellation-eligible and never becomes financially expiry-eligible. An admitted bond whose decision deadline has passed without a timely relationship decision is logically expiry-eligible even if an expiry worker has not yet materialized its terminal journal entries.

Define the unadmitted reserved set

$$\mathcal{R}_{ij} = \left\{ b \mid \begin{array}{l} b.\text{sender} = I_i, \ b.\text{recipient} = I_j, \\ b.\text{state} = \text{reserved} \end{array} \right\}. \quad (6)$$

For canonical receipt time t , define the decision-open admitted set

$$\mathcal{D}_{ij}(t) = \left\{ b \mid \begin{array}{l} b.\text{sender} = I_i, \ b.\text{recipient} = I_j, \\ b.\text{state} = \text{admitted}, \\ t \leq b.\text{decision_deadline} \end{array} \right\}. \quad (7)$$

Relationship decisions financially settle bonds in $\mathcal{D}_{ij}(t)$. Bonds in $\mathcal{R}_{ij}$ are cancelled with a full return because no message was admitted. An admitted bond whose deadline precedes t is logically expired and is in neither set.

6.6 Persistence Reserve

A persistence reserve **MUST** contain a unique identifier, (P_i, I_j) reference, the sender public-identity or directed-relationship reference, attempt and quote references, amount L_k , creation time, scheduled release time, state, and version. Its state is **reserved** or **released**.

L_k remains sender property. It **MUST NOT** be transferred to a recipient or provider. Cancellation before admission releases it immediately. Acceptance releases every active reserve associated with attempts from the accepted public identity to the recipient identity; it **MUST NOT** release a reserve associated with another public identity controlled by the same principal. Following rejection, blocking, or expiry, it remains reserved until its quoted release time and is then returned exactly once.

6.7 Native Message Envelope and Encrypted Content

Plaintext message content is not protocol or ledger state. In the native privacy profile, content comprises the subject, body, inline media, attachments, and any content-derived search representation. It is encrypted by the sender endpoint and decrypted only by authorized recipient endpoints.

The native message envelope contains only what is necessary to authenticate and route the attempt: a versioned opaque message identifier, scoped sender and recipient references, creation

and validity times, an encrypted-content reference, and the permission, capability, quote, bond, or admission proofs required for that delivery. It **MUST NOT** contain plaintext content, legal identity evidence, unrelated protocol identities, an account balance, or relationship history.

6.8 Relationship Solicitation State

The recipient provider **MUST** maintain or deterministically derive solicitation state for each directed unaccepted relationship. An active episode minimally contains an opaque episode identifier, the directed relationship handle, a monotonically increasing generation, associated admitted-attempt references, its open time, status, version, and the canonical position of its most recent change.

When admission finds no active episode, it **MUST** open the next generation and create exactly one authenticated logical relationship-solicitation effect. When admission finds an active episode, it **MUST** associate the attempt with that episode and **MUST NOT** create another relationship-establishment effect. It **MAY** update a grouped message count, unread state, or other quiet presentation.

The logical effect **MUST** use scoped references and **MUST NOT** contain plaintext message content, a legal identity, an account balance, or unrelated relationship history. Recipient settings may choose its delivery channel or suppress an interruptive push, but the active grouped solicitation remains available to the recipient.

Acceptance, rejection, and blocking **MUST** close the active episode. Expiry of the final decision-open admitted bond in an episode **MUST** cause the episode to lapse once the journal can determine that no timely decision applies. Closing or lapsing an episode moves no value by itself and does not change the settlement of any bond. Recipient presentation details other than the single-solicitation constraint remain outside the deterministic kernel.

7 Financial Semantics

The protocol bond is

$$B = C + S, \tag{8}$$

and the total attempt reservation is $R_k = B + L_k$. Ledger entries **MUST** use double-entry accounting or an equivalent representation that proves value conservation. Available balance, reserved balance, and owned balance **MUST** be distinguished.

The ledger journal **MUST** be append-only. Corrections **MUST** be represented by new compensating entries authorized by a separate rule; historical entries **MUST NOT** be rewritten. No compensation rule **MAY** silently violate terminal bond finality.

Protocol settlement defines ownership changes. External deposits, withdrawals, chargebacks, or bank transfers do not alter the meaning of a protocol transition. An implementation **MUST** prevent external funding failure from producing an unbacked valid reservation.

8 Commands and Transitions

The command subsections follow one reading pattern even where the normative text uses prose rather than identical headings:

1. **Authority and inputs:** who may issue the command and which canonical facts it carries;

2. **Preconditions:** versions, time, state, signatures, and funds that must be valid before commitment;
3. **Atomic outcome:** the complete state and ownership result;
4. **Idempotency and concurrency:** how replay or a stale snapshot is resolved; and
5. **Errors:** the stable failure described in Section 10.

The following table is an index, not a replacement for the normative command definitions:

Command	Principal gate	Atomic protocol result
IssueContactTerms	Current relationship, policy, and attempt state	Signed fixed terms, or a no-bond result for an accepted relationship.
ReserveAttempt	Valid current quote and sufficient available value	Reserve value and schedule the short admission deadline.
AdmitAttempt	Admission window, durable content, and exact reservation verified	Admit the message, start its decision window, advance attempt state, and open or join one solicitation.
CancelReserved Attempt	Authenticated sender cancellation, admission timeout, or pre-admission failure	Return $C + S + L_k$ in full and do not advance attempt state.
AcceptRelationship	Authenticated recipient decision at canonical time	Accept the public-identity relationship, settle admitted bonds, and fully cancel unadmitted reservations.
RejectRelationship	Authenticated recipient decision while unaccepted	Decline the solicitation, settle decision-open admitted bonds, and fully cancel unadmitted reservations.
BlockRelationship	Authenticated recipient decision for a directed pair	Prohibit future ordinary contact, settle admitted bonds, and fully cancel unadmitted reservations.
UnblockRelationship	Authenticated recipient controls a blocked relationship	Return to rejected status without accepting, resetting backoff, or reopening bonds.
ExpireBond	Deadline closed and no timely decision applies	Expire once, transfer C , return S , and retain L_k until release.
ReleasePersistence Reserve	Scheduled release time reached	Return L_k exactly once.
RevokeRelationship	Authenticated recipient controls an accepted relationship	Revoke future bond-free communication without reopening settled bonds.

8.1 Issue Contact Terms

IssueContactTerms evaluates the authenticated sender identity, current relationship, repeated-attempt state, recipient policy, and provider policy.

If $A_{ij} = \text{accepted}$, the provider **SHOULD** return a no-bond result and **MUST NOT** require C , S , or L_k .

If $A_{ij} = \text{blocked}$, the provider **MUST** return **ContactBlocked** and **MUST NOT** issue ordinary contact terms. This rule takes precedence over a previously issued scoped capability for the pair.

For **unknown**, **rejected**, or **revoked**, the provider **MUST** return signed terms consistent with the current relationship and repeated-attempt versions.

For an initial attempt, $k = 0$, $\Delta_0 = 0$, and $L_0 = 0$. Protocol policy **MUST** define nondecreasing schedules

$$\Delta_{k+1} \geq \Delta_k, \quad L_{k+1} \geq L_k. \quad (9)$$

8.2 Reserve Attempt

ReserveAttempt consumes a valid quote and an idempotency key. It **MUST**:

1. verify the quote signature, version, identities, expiry, and settlement unit;
2. verify that the relationship and repeated-attempt versions still match;
3. verify that the relationship remains bond-required;
4. verify that the sender has at least R_k available;
5. atomically reserve exactly $C + S$ as a bond and L_k as a separate persistence reserve;
6. compute the admission deadline from canonical reservation time and the quoted admission-window duration using checked arithmetic;
7. schedule idempotent cancellation at that deadline; and
8. return authenticated reservation evidence.

A stale quote **MUST** fail without retaining value. If acceptance linearized before this command, the command **MUST** return **RelationshipAccepted**; any provisional hold **MUST** be released. If blocking linearized before this command, it **MUST** return **ContactBlocked** and release any provisional hold.

8.3 Admit Attempt

AdmitAttempt verifies the message envelope, quote, bond, persistence reserve, current relationship, repeated-attempt state, and durable encrypted- content evidence. For an unaccepted relationship it **MUST** atomically:

1. confirm that the current time is not earlier than the quoted eligibility time;
2. confirm that canonical admission time is not after the bond's admission deadline;
3. confirm that exactly $C + S + L_k$ is reserved under the quoted terms;
4. confirm that the encrypted content and a retryable delivery intent can be committed durably with this transition;
5. change the bond from **reserved** to **admitted**;
6. record canonical admission time and compute the immutable decision deadline from the quoted decision-window duration;
7. cancel the scheduled unadmitted-reservation timeout and schedule bond expiry;
8. advance the repeated-attempt level;
9. establish the next eligibility time; and
10. open a new relationship solicitation episode and enqueue its single relationship-establishment effect, or join the existing active episode without another such effect; and

11. record authenticated admission evidence.

Admission is the boundary after which ordinary delivery retries do not reverse the protocol attempt. Advancement occurs on admission, not later settlement. Duplicate admission with the same idempotency key **MUST** return the original result. A conflicting replay **MUST** fail.

If the relationship became accepted after reservation but before admission, the reserved attempt **MUST** be cancelled unadmitted and its full $C + S + L_k$ returned. The message **MAY** then be admitted without a bond under ordinary accepted-sender policy. It **MUST NOT** advance repeated-attempt state or open an unaccepted- relationship solicitation episode.

If the relationship became blocked after reservation but before admission, the block transition has already cancelled the unadmitted reservation and returned $C + S + L_k$ in full. Admission **MUST** return **ContactBlocked**, create no message or solicitation effect, and perform no additional value movement.

8.4 Accept Relationship

AcceptRelationship is authenticated by the recipient and targets (I_i, I_j) . Let t_a be its authoritative receipt time. In one atomic transition, the command **MUST**:

1. set $A_{ij} \leftarrow \text{accepted}$ and increment the relationship version;
2. identify every bond in $\mathcal{D}_{ij}(t_a)$ and mark it **accepted**;
3. release exactly $C + S$ for each accepted bond to its sender and cancel its scheduled expiry;
4. identify every bond in $\mathcal{R}_{ij}$, mark it **cancelled_unadmitted**, return its complete $C + S + L_k$, and cancel its admission-timeout operation;
5. release every other active persistence reserve associated with an attempt from I_i to I_j , including reserves for earlier expired or rejected attempts;
6. preserve the repeated-attempt level and backoff for (P_i, I_j) and preserve every reserve associated with another public identity controlled by P_i ;
7. close the active relationship solicitation episode and suppress any not-yet-delivered relationship-establishment effect for it; and
8. append the relationship, solicitation, bond, reserve, and ledger events as one atomic protocol outcome.

Acceptance settles the relationship, not merely the message beside which the recipient acted. No bond included in the acceptance transition **MAY** later be rejected or expired. No message from I_i admitted after the transition **MAY** require a bond while the relationship remains accepted.

A bond whose decision deadline precedes t_a is not in $\mathcal{D}_{ij}(t_a)$. It **MUST** settle as expired if it has not already done so. The acceptance still grants future permission and settles every other decision-open admitted bond. It **MUST NOT** reopen a terminal or logically expired bond.

Acceptance of I_i **MUST NOT** accept another public identity controlled by P_i , reset the principal-level repeated-attempt state, or release persistence reserves associated with attempts from another public identity.

8.5 Reject Relationship

RejectRelationship is authenticated by the recipient and targets (I_i, I_j) . It is invalid while $A_{ij} = \text{accepted}$; an accepted relationship **MUST** first be revoked. Let t_r be the authoritative receipt time. In one atomic transition, rejection **MUST**:

1. set $A_{ij} \leftarrow \text{rejected}$ and increment the relationship version;
2. identify every bond in $\mathcal{D}_{ij}(t_r)$;
3. mark each identified bond **rejected**;
4. transfer each identified bond's C to the recipient provider;
5. transfer each identified bond's S to the recipient;
6. cancel scheduled expiry operations for those bonds;
7. identify every bond in $\mathcal{R}_{ij}$, mark it **cancelled_unadmitted**, return its complete $C + S + L_k$, and cancel its admission-timeout operation;
8. preserve the advanced repeated-attempt level and backoff; and
9. retain each L_k associated with an admitted bond settled by rejection until its quoted release time; and
10. close the active relationship solicitation episode and suppress any not-yet-delivered relationship-establishment effect for it.

Rejection settles every decision-open admitted bond for the relationship at its canonical ordering point and fully cancels every unadmitted reservation. A selected message is evidence of which relationship the recipient means; it does not limit settlement to that message. A bond settled by rejection **MUST NOT** later be accepted or expired.

Rejection is not a permanent contact prohibition. The rejected state remains bond-required, preserves the repeated-attempt level and backoff, and permits a later eligible sender attempt to obtain new terms and open a new solicitation.

8.6 Block Relationship

BlockRelationship is authenticated by the recipient and targets (I_i, I_j) . It **MAY** be applied from any relationship state. Let t_b be its authoritative receipt time. In one atomic transition, blocking **MUST**:

1. set $A_{ij} \leftarrow \text{blocked}$ and increment the relationship version;
2. identify every bond in $\mathcal{D}_{ij}(t_b)$;
3. settle each identified bond under the rejection allocation by marking it **rejected**, transferring C to the recipient provider, and transferring S to the recipient;
4. cancel scheduled expiry operations for those bonds;
5. identify every bond in $\mathcal{R}_{ij}$, mark it **cancelled_unadmitted**, return its complete $C + S + L_k$, and cancel its admission-timeout operation;

6. preserve the repeated-attempt level, backoff, and scheduled release of every L_k associated with an admitted bond settled by blocking;
7. invalidate scoped capabilities for the directed pair;
8. close the active relationship solicitation episode and suppress any not-yet-delivered relationship-establishment effect for it; and
9. append the relationship, solicitation, bond, capability, reserve, and ledger events as one atomic protocol outcome.

While the relationship remains blocked, the recipient provider **MUST NOT** issue ordinary contact terms, accept a reservation, or admit an ordinary or capability-authorized attempt for the pair. An exact replay returns the recorded result without another settlement.

8.7 Unblock Relationship

UnblockRelationship is authenticated by the recipient. It **MUST** change A_{ij} from **blocked** to **rejected** and increment the relationship version. It **MUST NOT** accept the relationship, reset repeated-attempt state, release persistence reserves early, restore invalidated capabilities, reopen settled bonds, or create a solicitation. A future sender attempt remains subject to the preserved backoff and must obtain fresh contact terms.

8.8 Expire Bond

Only an admitted bond has a decision deadline T_b and may be expired. A reserved bond is governed by its admission deadline and **CancelReservedAttempt**. A decision is timely for admitted bond b when its authoritative receipt time is at or before T_b . Expiry **MUST NOT** become terminal until the canonical journal can determine that no timely acceptance, rejection, or blocking decision applies to the bond.

An implementation **SHOULD** use a bounded processing holdback after T_b before materializing expiry. The holdback is not an extension of the recipient's decision window; it allows decisions already durably received by T_b to be sequenced and observed. The quote or provider policy **MUST** disclose the earliest time at which expiry settlement may be considered final.

If a timely acceptance exists, acceptance settlement **MUST** win. If a timely rejection exists and no earlier valid acceptance changed the relationship to accepted, rejection settlement **MUST** win. A timely block uses the rejection allocation and then prohibits future contact. Conflicting recipient decisions are ordered by the canonical journal and relationship-version preconditions.

If neither decision applies, **ExpireBond** **MUST** atomically:

1. mark the bond **expired**;
2. transfer C to the recipient provider;
3. release S to the sender;
4. leave the current relationship state unchanged;
5. preserve the repeated-attempt level and backoff; and
6. retain the associated L_k until its quoted release time; and

7. if no associated admitted bond remains decision-open, mark the active relationship solicitation episode lapsed without moving value.

The expiry command **MUST** re-read bond state and relevant relationship decisions inside its settlement transaction. A stale expiry job encountering an accepted, rejected, expired, or unadmitted-cancelled bond **MUST** produce no additional transfer.

Expiry normally leaves an unknown, rejected, revoked, or blocked relationship in that state. If a late acceptance has already established prospective permission, expiry of an older decision-closed bond **MUST NOT** undo that acceptance.

8.9 Cancel Reserved Attempt

CancelReservedAttempt applies only while the bond remains **reserved**. It may be authenticated by the sender as a voluntary cancellation, by the provider as a verified pre-admission technical failure, or by a scheduled command whose canonical time is after the admission deadline. In one atomic transition it **MUST**:

1. mark the bond **cancelled_unadmitted**;
2. return the full $C + S + L_k$ to the sender;
3. cancel the admission-timeout operation;
4. leave repeated-attempt state unchanged; and
5. create no message, solicitation, decision deadline, or delivery effect.

An exact replay returns the recorded result without another release. A scheduled timeout that encounters an admitted or terminal bond is a no-op. The sender **MAY** cancel before the admission deadline; at or after the deadline the provider **MUST** make the same full-return transition if the attempt remains reserved.

After admission, ordinary delivery retry is an outbox concern and **CancelReservedAttempt** is invalid. Admission therefore **MUST NOT** commit until encrypted content and a retryable delivery intent are durable. A later accounting correction requires a separately authorized compensating journal entry; it does not erase the admitted attempt or roll back repeated-attempt state.

8.10 Release Persistence Reserve

At or after a reserve's scheduled release time, an idempotent **ReleasePersistenceReserve** command **MUST** return L_k to the sender if the reserve remains active. If identity-scoped acceptance or unadmitted cancellation already released it, the command **MUST** return the original result without another ledger entry.

8.11 Revoke Relationship

RevokeRelationship is authenticated by the recipient. It **MUST** change A_{ij} from **accepted** to **revoked** and increment the relationship version. It **MUST NOT** alter previously settled bonds. Future ordinary messages require a new quote and bond unless a valid scoped capability authorizes them.

9 Concurrency Rules

9.1 Acceptance and Concurrent Reservations

Acceptance and bond reservation for the same relationship **MUST** serialize. A reservation ordered before acceptance is fully cancelled if it remains unadmitted, or belongs to the decision-open admitted set if admission also linearized before acceptance. A reservation ordered after acceptance **MUST** fail as unnecessary and release any provisional hold. Rejection and blocking use the same ordering rule, except that decision-open admitted bonds use the rejection allocation.

9.2 Admission and Unadmitted Cancellation

Admission and cancellation for the same reserved attempt **MUST** serialize. If admission commits at or before the admission deadline, it cancels the timeout, starts the decision window, and advances repeated-attempt state. If sender cancellation commits first, or the attempt remains reserved after the admission deadline, the full reservation returns and admission cannot later revive it. There is no state in which an attempt both advances repeated-attempt history and is cancelled as unadmitted.

9.3 Acceptance and Expiry

There is no undefined simultaneous outcome. If acceptance was durably received on or before a bond's deadline, it **MUST** take priority even when an expiry worker began first. If acceptance was received after the deadline, the bond expires and acceptance applies prospectively and to other bonds whose windows remain open. Once either settlement commits, retries of the losing command are no-ops with respect to value.

9.4 Conflicting Relationship Decisions

Conflicting acceptance, rejection, blocking, unblocking, or revocation commands **MUST** carry an expected relationship version or equivalent compare-and-set condition. The first command that commits changes the version. A conflicting stale command **MUST** fail with `VersionConflict` and **MUST NOT** alter any bond. A recipient who wishes to decline a newly accepted relationship uses revocation; a recipient who wishes to prohibit future contact uses blocking.

9.5 Concurrent Admissions and Solicitation

Admissions for the same unaccepted directed relationship **MUST** serialize on the solicitation generation or use an equivalent uniqueness constraint. Exactly one admission may change an inactive solicitation to active for a generation and create its relationship-establishment effect. Every concurrent admission ordered afterward joins that generation without creating another such effect. A retry of the winning admission returns its recorded result rather than opening a new generation.

9.6 Multiple Pending Bonds

Relationship settlement **MUST** enumerate or deterministically derive the complete decision-open admitted set and the complete unadmitted reserved set at the transition's journal position. Pagination, sharding, or asynchronous projections **MUST NOT** cause partial relationship settlement or leave an unadmitted reservation stranded. Implementations **MAY** use a transaction manifest or prepare/commit protocol, but the externally visible result **MUST** be atomic.

10 Error Conditions

An implementation **MUST** return a stable machine-readable error and **MUST NOT** retain or transfer funds except as explicitly stated below.

Condition	Required behavior
AuthenticationFailed	Reject the command without state or ledger effects.
InvalidSignature	Reject the quote, event, assertion, or receipt without state or ledger effects.
QuoteExpired	Refuse new reservation; release any provisional hold.
QuoteVersionStale	Refuse reservation or admission and require new contact terms.
RelationshipAccepted	Do not create a bond; release any provisional hold and continue only under accepted-sender policy.
ContactBlocked	Do not issue terms, reserve value, or admit an ordinary or capability-authorized attempt for the blocked directed pair. Release any provisional hold.
BackoffActive	Refuse admission and report the next eligible time; do not advance the attempt level.
InsufficientFunds	Create no bond or persistence reserve.
AmountMismatch	Refuse reservation or admission; no underfunded bond is valid.
AdmissionWindowClosed	If the bond remains reserved, cancel it, return $C + S + L_k$ in full, and do not admit or advance attempt state.
ReservedAttemptCancelled	Return the recorded full-release result; do not admit, create a solicitation, or move value again.
DecisionWindowClosed	Do not apply the decision to the closed bond; the relationship command MAY still have prospective effect.
AlreadyTerminal	Return the recorded terminal result without another transfer or release.
VersionConflict	Apply no partial relationship or ledger effects; the caller MUST refresh state before another command.
Duplicate	Return the original result when payload and idempotency key match. A conflicting payload using the same key MUST fail.
PrivacyScopeViolation	Refuse an identifier translation, disclosure, or query that exceeds the caller's authorized relationship and purpose.
ContentKeyUnavailable	Do not ask a provider to bypass end-to-end encryption. Return ciphertext or an endpoint recovery error without changing settlement.
LinkageErased	Report that expired identifying linkage cannot be reconstructed; do not restore it from analytics, logs, or an unrelated backup.

Provider timeouts, database retries, duplicate network delivery, and process crashes **MUST** resolve to one of these deterministic outcomes. An unknown outcome **MUST** be queried by idempotency key before the caller retries with a new key.

11 Launch Extension: Scoped Capabilities and Express Lanes

The base settlement kernel remains usable without scoped capabilities. The reference deployment advertises the express-lane capability extension at launch; an implementation that does not advertise it treats unaccepted ordinary communication under the bond rules above. When advertised, the following requirements apply.

A recipient **MAY** issue a signed communication capability containing scoped sender and recipient references, purpose or message class, validity interval, maximum count or rate, capability version, and revocation conditions. Outside the endpoint and routing boundary, those references **MUST** be pairwise opaque.

An advertised express-lane capability **MUST** contain a recipient signature, a hard validity interval, and an enforceable maximum count or rate. Purpose labels **MUST NOT** be represented as provider-enforced content policy when content is end-to-end encrypted. A native opaque pair handle identifies the subject but is not authority without the signed grant.

For a legacy sender, a DMARC pass under RFC 9989 is evidence that use of the RFC 5322 author domain was authorized; it is not evidence that a message is safe or that a look-alike domain belongs to an expected organization. A legacy lane **MUST** bind separately to a canonical IDNA-ASCII organizational domain and **MUST** apply the same versioned domain-to-protocol-identity mapping for grant, admission, blocking, and revocation. The aligned DKIM or SPF identifier **MUST NOT** silently replace that lane identity.

A message matching a valid capability **MAY** be admitted without a bond while the relationship remains unaccepted. Capability validation **MUST** be deterministic, auditable, scoped to the represented authority, and replay-safe. A capability **MUST NOT** silently create unrestricted standing acceptance. A capability-authorized message **MUST NOT** open or join a relationship solicitation and **MUST NOT** advance repeated-attempt state. A **blocked** relationship overrides a capability for the directed pair; blocking **MUST** invalidate that authority until the recipient explicitly issues new authority after unblocking.

12 Security, Privacy, and Governance Requirements

12.1 Authentication and Replay

Every quote, reservation assertion, admission receipt, recipient decision, settlement receipt, and provider assertion **MUST** be authenticated. Every state-changing command **MUST** carry an idempotency key and domain separation sufficient to prevent reuse in another command type, relationship, or protocol version.

Operational keys **MUST** be rotatable without changing P_i . Recovery and key revocation **MUST** prevent a compromised accepted identity from retaining indefinite bond-free authority.

12.2 Native Content Confidentiality

A conforming native client **MUST** encrypt message content before transmitting it to a provider. Only sender-authorized and recipient-authorized endpoints **MAY** hold the ordinary content-decryption keys. A provider, ledger operator, content store, or delivery worker **MUST NOT** require possession of those keys as a condition of native delivery.

The cryptographic binding between the encrypted content and its authenticated envelope **MUST** prevent substitution, truncation, reordering, and attachment replacement. The recipient **MUST** be able to authenticate the sending protocol identity and detect an unexpected operational-key change.

Implementations **SHOULD** provide auditable key transparency or an equivalent mechanism that makes silent key substitution detectable.

Content storage **MUST** use ciphertext references rather than embedding content in relationship or ledger events. Server-side indexes **MUST NOT** contain plaintext subjects, bodies, attachments, or content-derived terms. Search over native content **MUST** occur at an authorized endpoint or through a construction that does not disclose the searched plaintext to the provider.

An SMTP gateway or other legacy bridge **MAY** necessarily encounter plaintext or provider-readable content. Such delivery **MUST** be identified as outside the native content-confidentiality guarantee. A provider **MUST NOT** represent ordinary TLS transport or provider-managed encryption at rest as native end-to-end encryption.

12.3 Pairwise Relationship Privacy

The logical relationship remains (I_i, I_j) , but components that do not route or display an address **MUST** represent it by an opaque pairwise handle H_{ij} . The handle **MUST** be generated randomly or by a secret-key construction with domain separation and at least 128 bits of effective unpredictability. It **MUST NOT** be a plain, unsalted, or otherwise publicly testable hash of the two protocol identities.

A handle is scoped to one directed identity pair and one service domain. The handle used for I_i contacting I_j **MUST NOT** reveal the handle used for the same sender contacting another recipient. The reverse relationship from I_j to I_i **MUST** use a distinct handle. Relationship, content, ledger, and observability systems **SHOULD** use separately scoped references so that one global identifier does not join all records without passing through a controlled translation boundary.

The private repeated-attempt relationship (P_i, I_j) similarly **MUST** be represented outside the identity service by a recipient-scoped opaque assertion. That assertion **MUST** allow the correct anti-persistence history to survive changes of sender address, key, or provider, while preventing unrelated recipients from comparing assertions to discover a common private principal.

A provider migration **MUST NOT** export a globally stable principal identifier. Where continuity across providers is supported, the prior identity authority **MUST** instead issue or authorize a migration proof scoped to the recipient and the repeated-attempt purpose. The receiving provider **MUST NOT** be able to use that proof to discover the principal's relationships with other recipients.

There **MUST NOT** be a public or ordinary administrative interface that enumerates the global relationship graph. A recipient **MAY** inspect its own visible relationships, and a sender **MAY** inspect its own attempts and receipts, but neither party receives another participant's unrelated edges. Pending, rejected, ignored, revoked, blocked, and accepted relationships receive the same graph-confidentiality protection.

An accepted relationship **MAY** persist as an opaque handle or recipient-issued capability together with current state, version, and revocation information. The system **MUST NOT** retain the complete formation history merely because current permission remains active.

12.4 Compartmentalization and Minimal Disclosure

The implementation **MUST** separate identity evidence, relationship state, encrypted content, financial settlement, and operational telemetry into distinct access domains. This is a separation of authority as well as storage. Access to one domain **MUST NOT** automatically grant access to the keys or translation data of another.

Within ordinary operation, the identity service **MAY** know that a verified principal controls a public identity, but it need not receive message content. The relationship service **MAY** know

H_{ij} and its state, but it need not receive legal identity evidence or plaintext content. The ledger **MAY** know opaque account, bond, and settlement references together with amounts, but it need not receive a subject, content reference, public address pair, or private principal identifier. The content store **MAY** know a ciphertext object and a scoped retrieval reference, but it need not receive a bond outcome or readable endpoint identities.

The delivery boundary necessarily learns enough to route to I_j and to present I_i to the recipient. It **MUST** disclose no additional principal evidence, unrelated identities, balance information, or relationship history. A recipient provider **MUST** verify the applicable reservation or capability, but it **MUST NOT** receive the sender's overall balance or unrelated settlement activity. External financial rails **SHOULD** see account funding and withdrawal rather than individual message events. Internal transfers **SHOULD** be netted or aggregated before external settlement when correctness and applicable obligations permit.

Translation among scoped references is itself sensitive processing. Every translation **MUST** be authorized for a specific protocol purpose and recorded in an access audit that does not reproduce the translated graph. Bulk translation, graph export, and use of relationship data for advertising, sale, or unrelated profiling are outside the protocol purpose and **MUST NOT** be enabled by default.

12.5 Metadata Leakage and Traffic Analysis

Pairwise handles do not conceal every observable fact. Timing, network addresses, message sizes, uncommon bond amounts, and synchronized journal events can permit correlation without a direct identifier. Implementations **MUST** document which of these signals each actor can observe and **MUST** retain them only while needed for routing, security, settlement, or a disclosed obligation.

Where it does not compromise correctness, an implementation **SHOULD** reduce correlation through coarse timestamps, batching, settlement netting, content padding, separation of network and account logs, and short retention of delivery telemetry. These protections **MUST NOT** alter quoted amounts, obscure a settlement from its owner, or make bond finality ambiguous.

12.6 Record Lifecycle and Retention

Every stored field **MUST** belong to a documented retention class stating its purpose, access domain, controlling key, operational lifetime, dispute lifetime, any separately applicable required-retention period, and deletion method. A vague interest in future analytics is not a protocol purpose for indefinite retention.

While an attempt is pending, the protocol **MAY** retain the scoped linkage required to authenticate admission, prevent replay, apply repeated-attempt policy, receive a decision, and settle the bond. Once the attempt is terminal and its defined dispute period ends, the implementation **MUST** remove fields and indexes that are not needed for continuing permission, accounting, fraud handling, or an applicable retention obligation. A durable accounting record **SHOULD** contain only an opaque settlement reference, amount, settlement unit, terminal state, canonical time, and the minimum ownership evidence required for audit.

Continuing acceptance does not justify continuing retention of individual contact attempts. After acceptance, the live relationship **MAY** retain H_{ij} , current state, version, capability or key references, and revocation material. Formation messages, quote details, delivery telemetry, and superseded relationship events **MUST** follow their own shorter retention schedules unless independently needed.

Records retained solely for a legal, accounting, fraud, or dispute obligation **MUST** be segregated from online delivery and product analytics. They **MUST** use separate encryption keys, narrowly

authorized access, an explicit expiration condition, and an audit trail. A hold on one record or account **MUST NOT** silently extend retention for unrelated users or relationships.

Deletion **MUST** cover primary storage, replicas, caches, search indexes, telemetry, derived datasets, and backups according to a documented propagation schedule. If immediate removal from an immutable backup is impracticable, the backup **MUST** remain inaccessible to ordinary processing, expire within a fixed period, and lose access to deleted record keys no later than that period.

12.7 Cryptographic Erasure and Consent-Locked Archives

Encryption is not erasure while the operator retains both a usable key and enough linkage to identify the parties. When a retention period ends, the implementation **SHOULD** delete the unnecessary record and destroy every operator-held key that could recover its identifying linkage. A claim of cryptographic erasure is valid only when replicas, wrapped key copies, recovery escrow, and backups can no longer restore that linkage.

The protocol **MAY** support a consent-locked historical archive in which reconstruction requires independent sender and recipient key shares. Such an archive **MUST** be separate from live permission and settlement state. Refusal or loss of a share **MUST NOT** prevent the protocol from delivering under current permission, applying revocation, enforcing backoff, or completing a pending settlement.

If an implementation offers two-party reconstruction, it **MUST** state precisely which fields it covers and whether any separately retained compliance record can identify the transaction without both shares. It **MUST NOT** promise two-party-only reconstruction for information that the operator knowingly retains under an independent decryption key. Required records that cannot be consent-locked **MUST** be minimized and isolated under the preceding retention rules.

No deletion or threshold scheme can erase a message or receipt independently kept by a sender, recipient, endpoint backup, or external provider. The protocol's guarantee concerns records controlled by conforming providers and **MUST** describe that boundary accurately.

12.8 Operational Logs and Analytics

Logs **MUST NOT** contain plaintext content, private identity evidence, raw key material, full account balances, or a reusable global relationship identifier. Debugging and tracing identifiers **MUST** be scoped and rotated so that an observability system does not become an accidental relationship graph.

Analytics **SHOULD** operate on aggregates produced after the shortest useful window. A dataset is not anonymous merely because names and addresses have been removed; if scoped handles, rare amounts, precise timestamps, or auxiliary data permit practical reidentification, it remains protected relationship data and **MUST** follow the corresponding access and retention rules.

12.9 Provider and Recipient Incentives

A provider receiving C has an economic interest in rejection or expiry. A recipient receiving S may have an interest in attracting and rejecting bonded contact. Deployments **MUST** make settlement auditable and **MUST** define fraud, collusion, reversal, suspension, and dispute policies outside the deterministic kernel.

Recipient providers **MUST** present acceptance as an effective available action, **MUST** provide signed decision receipts, and **MUST NOT** suppress a timely acceptance while claiming expiry.

Pricing policy **SHOULD** define bounds or review mechanisms for C and provider-imposed minimum S .

12.10 Delivery Meaning

A valid bond authorizes a protocol-valid contact attempt. It does not guarantee inbox prominence, human attention, a response, or exemption from malware, harassment, unlawful-content, or safety controls. Providers **MUST** distinguish financial validity from content and transport policy.

Those controls do not authorize provider decryption of native content. They **MAY** operate on the authenticated envelope, endpoint evaluation, recipient reports, rate and relationship state, or content a participant deliberately discloses for investigation. Any disclosed plaintext **MUST** be scoped to that purpose and its own retention class.

13 Interoperability Requirements

A federated implementation **MUST** provide versioned representations for:

- provider and protocol identities;
- contact-term quotes;
- message envelopes and content references;
- bond and persistence-reserve commitments;
- repeated-attempt assertions;
- admission and unadmitted-cancellation receipts;
- authenticated relationship decisions;
- settlement receipts; and
- provider-to-provider clearing obligations.

Federation **MUST** preserve the native privacy profile across provider boundaries. A provider assertion identifying P_i for repeated-attempt enforcement **MUST** be scoped to I_j or its recipient-provider privacy domain and **MUST NOT** provide a stable value that other recipients can compare. Settlement and admission receipts **MUST** use the minimum scoped references needed by their verifiers. A downgrade to SMTP, provider-readable content, or a less private metadata profile **MUST** be shown to the sender before transmission and represented in the message envelope.

The sender provider **MUST** prove reservation of the quoted $C + S + L_k$. The recipient provider **MUST** verify that proof and the current quote state before admission. Inter-provider clearing transfers obligations created by C and S ; L_k remains sender property and **MUST NOT** enter provider revenue or net clearing.

Independent implementations **MAY** use different storage engines, internal ledger software, and external financial rails, provided that identical canonical inputs produce equivalent protocol and ownership transitions.

14 Required Invariants

Every conforming implementation **MUST** preserve all of the following:

1. **Bond finality.** Each bond reaches at most one terminal state.
2. **Reserve finality.** Each persistence reserve is released exactly once and is never transferred to a recipient or provider.
3. **Value conservation.** Every transition conserves ledger value.
4. **Exact funding.** An unaccepted ordinary attempt is not valid unless exactly $C + S + L_k$ is reserved under valid fixed terms.
5. **Relationship-wide acceptance.** Acceptance atomically establishes permission for (I_i, I_j) , refunds every decision-open admitted bond, fully cancels every unadmitted reservation for the pair, and releases active persistence reserves associated with attempts from I_i to I_j .
6. **No later rejection.** A bond or message included in acceptance cannot subsequently be rejected or expired.
7. **Relationship-wide rejection.** Rejection atomically settles every decision-open admitted bond for (I_i, I_j) according to the rejection allocation and fully cancels every unadmitted reservation for the pair.
8. **Retryable rejection.** Rejection preserves repeated-attempt backoff and permits a later eligible bonded attempt; it is not a block.
9. **Recipient blocking.** Blocking settles every decision-open admitted bond under the rejection allocation, fully cancels every unadmitted reservation, and prevents terms, reservation, capability use, and admission for the directed pair until authenticated unblocking.
10. **Unadmitted full return.** Cancellation before admission returns exactly $C + S + L_k$, creates no solicitation or decision deadline, and does not advance repeated-attempt state.
11. **Timely-decision priority.** A decision durably received by a bond's deadline is observed before expiry becomes final.
12. **No retroactive reopening.** A later acceptance may grant future permission but cannot reopen a terminal or logically expired bond.
13. **Atomic attempt advancement.** Admission and repeated-attempt advancement occur atomically.
14. **Principal continuity.** Changing a sender address, key, or provider does not reset (P_i, I_j) attempt state.
15. **Identity-scoped permission.** Acceptance of one public identity does not accept other identities controlled by the same principal, reset principal-level attempt state, or release their persistence reserves.
16. **Principal non-exportability.** A private principal reference is not a public address or globally comparable inter-provider identifier; other domains receive only purpose-limited, scoped assertions.

17. **Free accepted communication.** Accepted ordinary communication requires no protocol bond and incurs no protocol message fee.
18. **Replay safety.** Duplicate commands, events, receipts, and expiry jobs cannot duplicate value movement or state transition.
19. **Atomic visibility.** No observer may see permission change without its required bond and reserve settlements, or vice versa.
20. **Native content blindness.** Providers cannot decrypt a native message’s subject, body, inline media, or attachments with provider-controlled keys.
21. **Pairwise unlinkability.** An opaque handle or principal assertion disclosed for one recipient cannot be used by another recipient to recognize the same relationship or private principal.
22. **Graph non-enumerability.** No public or ordinary administrative operation exposes the global relationship graph or contact-attempt history.
23. **Data compartmentalization.** Routine access to content, identity, relationships, the ledger, or telemetry does not automatically grant access to the other privacy domains or their translation keys.
24. **Bounded retention.** Every identifying field has a declared purpose, retention class, expiration condition, and deletion path across primary and derived storage.
25. **Honest erasure.** A record described as cryptographically erased cannot be reconstructed from operator-held keys, key escrow, replicas, logs, or backups.
26. **Rail minimization.** External funding and withdrawal systems do not receive message content or a per-message relationship history from the protocol.
27. **Solicitation coalescing.** One active solicitation generation for a directed unaccepted relationship produces exactly one logical relationship- establishment effect; later admitted attempts join it without another such effect.
28. **Attempt distinctness.** Solicitation coalescing does not merge bonds, deadlines, content, persistence reserves, or attempt advancement, and closing an episode does not itself move value.

15 Reference Transition Summary

For all bonds in the applicable decision-open admitted set, the relationship outcomes are:

$$\begin{aligned}
\text{Accept relationship} & : C + S \rightarrow \text{sender}, \\
\text{Reject relationship} & : C \rightarrow \text{recipient provider}, \quad S \rightarrow \text{recipient}, \\
\text{Block relationship} & : C \rightarrow \text{recipient provider}, \quad S \rightarrow \text{recipient}, \\
\text{Expire bond} & : C \rightarrow \text{recipient provider}, \quad S \rightarrow \text{sender}.
\end{aligned} \tag{10}$$

Every unadmitted reservation affected by cancellation or a relationship decision returns $C + S + L_k$ in full. Acceptance releases every other active L_k associated with attempts from the accepted I_i to I_j and preserves principal-level attempt state. Rejection, blocking, and expiry retain the L_k associated with admitted attempts until their scheduled release. The first admitted attempt in a

solicitation generation creates exactly one logical relationship-establishment effect. Later attempts join that generation without a new solicitation. Acceptance, rejection, or blocking closes it; absence of a decision leaves each admitted bond to expire separately and the episode lapses when none remains decision-open. Rejection permits another eligible attempt after backoff; blocking prohibits one until the recipient unblocks.

16 Implementation Sequence

A reference implementation **SHOULD** proceed in the following order:

1. implement the in-memory deterministic relationship, bond, reserve, and repeated-attempt transitions;
2. test full event sequences, concurrent decisions, and the required invariants with unit and property-based tests;
3. add the append-only double-entry journal and idempotent settlement;
4. add signed identities, endpoint content encryption, key transparency, quotes, admission receipts, and decision receipts;
5. introduce pairwise handles and enforce the identity, relationship, content, ledger, and telemetry access boundaries;
6. persist journal streams and materialized projections with retention classes, key separation, and tested cryptographic erasure;
7. add delivery, encrypted content storage, solicitation coalescing, and expiry processing;
8. add express-lane scoped capabilities without changing base settlement, relationship, or repeated-attempt semantics;
9. add visibly downgraded SMTP compatibility gateways; and
10. add privacy-preserving federation and clearing after centralized semantics are stable.

The first milestone is complete only when it can deterministically execute quote, reservation, admission, relationship-wide acceptance and rejection, expiry, unadmitted cancellation and full release, deferred reserve release, principal-level attempt-state preservation, revocation, blocking, unblocking, and the opening, joining, and closing of solicitation episodes while preserving every invariant in this document. Scoped capabilities are not required inside this settlement milestone. A production milestone additionally requires native end-to-end content encryption, pairwise relationship references, compartmentalized access, and verified deletion across replicas and backups.